



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

VENDING MACHINE CONTROLLER

The domain of the Project:

RTL Design Verification

Team Mentors:

Sri. Satish Devarapalli

Sri. Chetan

Team Members:

- 1. Mr. Dhanush H** (B.E ECE 4th Year) Team Leader
- 2. Mr. Shankar J** (B.Tech ECE Graduate-2024)
- 3. Ms. Golla Sailaja G** (B.tech ECE Graduate-2024)
- 4. Mr.Saiteja B** (B.Tech ECE Graduate - 2024)

Period of the project

July 2024 to April 2025



SURE TRUST

PUTTAPARTHI, ANDHRA PRADESH



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

DECLARATION

The project titled “*Vending Machine Controller*” has been mentored by **Sri. Satish Devarapalli, Sri. Chetan** and organised by SURE Trust, from July 2024 to April 2025, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

H. Dhanush.

Signature

Mr. Dhanush H (Lead)

Shankar. J

Signature

Mr. Shankar J

B. Saiteja.

Signature

Mr. Saiteja B

G. Sailaja

Signature

Ms. Golla Sailaja G

Team Mentors:

Signature

Sri. Satish Devarapalli

Signature

Sri. Chetan

Seal & Signature

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Table of Contents

1. DECLARATION	ii
2. TABLE OF CONTENTS	iii
3. EXECUTIVE SUMMARY	1
4. INTRODUCTION	2
4.1 Background and Context	2
4.2. Problem Statement	2
4.3. Scope	3
4.4. Limitations	4
4.5. Innovation	5
5. PROJECT OBJECTIVES	6
5.1. Project Objectives and Expected Outcomes	6
5.2. Deliverables	7
6. METHODOLOGY AND RESULTS	8
6.1. Methods/Technology Used	8
6.2. Tools/Software Used	8
6.3. Data Collection Approach	8
6.4. Project Architecture	9
6.5. Results	12
6.6. GitHub Link	15
7. LEARNING AND REFLECTION	16
7.1. Learning and Reflection	16
7.2. Experience	17
8. CONCLUSION AND FUTURE SCOPE	18
8.1. Objectives	18
8.2. Achievements	18
8.3. Conclusion	19
8.4. Future Scope	19



Executive Summary

In this project, the Vending Machine Controller is verified using a detailed **UVM (Universal Verification Methodology)** process to ensure it is performed correctly across all possible scenarios. A UVM-based testbench was created to simulate real-world conditions such as coin insertion, product selection, insufficient balance, and change return. Key verification components, including **stimulus generators, sequencers, agents, drivers, monitors, scoreboards, and coverage collectors**, were utilized to validate the design's behaviour. Assertions and self-checking mechanisms were implemented to monitor the functionality at each stage, while functional coverage ensured that all states and transitions were tested. This project provided hands-on experience in **UVM-based verification**, simulation, and debugging, allowing me to develop practical skills aligned with industry-standard verification practices and roles.



Introduction

Background and Context of the Project:

As part of learning and exploring the verification field, we worked on verifying a Vending Machine Controller that was designed using Verilog HDL. Since vending machines operate based on a sequence of actions depending on user inputs, they are a good example to apply finite state machines (FSMs) and verification concepts. The main goal is to get hands-on experience in functional verification by writing a testbench that could check different situations like inserting coins, selecting products, and returning change. Through this, we learned how to create proper test environments, find and fix bugs, and make sure the design works correctly in all expected conditions.

Problem Statement:

The main goal of the project is to verify the working of a Vending Machine Controller using Universal Verification Methodology (UVM). The controller, supposed to handle different operations like detecting coins, selecting products, dispensing items, and giving change based on a finite state machine (FSM). The idea is to create a proper UVM testbench that could check all these scenarios, find any possible bugs, and make sure the design works as expected. Through this project, we wanted to get hands-on experience with UVM components like sequencers, drivers, monitors, and scoreboards, and also improve our skills in debugging.



Scope

The scope defines the boundaries and goals of the project — specifically what is included in the project activities and objectives. It outlines what the verification environment aims to achieve.

1. **Functional Verification of the Vending Machine Controller** : The project focuses on validating the functionality of a digital IP core that simulates a vending machine, ensuring correct operations like coin insertion, product selection, dispensing, and change return.
2. **UVM-Based Verification Environment** : A modular testbench is developed using Universal Verification Methodology (UVM), allowing for scalable, reusable, and well-structured test components.
3. **FSM Behavior Testing** : The controller's FSM behavior is verified by checking valid operation sequences and expected transitions through states like RESET → CONFIG → OPERATION → DISPENSE → CHANGE, as defined in the spec.
4. **Use of UVM Components** : Incorporates key UVM blocks like sequences, drivers, monitors, agents, scoreboard, and coverage to create a complete and robust verification architecture.
5. **Functional Coverage Analysis** : Ensures comprehensive verification by collecting and analyzing coverage metrics, enabling detection of corner cases and untested scenarios.



Limitations

Limitations refer to the constraints and boundaries that restrict the full realization or generalization of the project. These may be due to time, resource, design, or environment factors.

1. **Time Constraints for Full Verification** : Due to limited project duration, only core functional paths are thoroughly verified, while exhaustive corner-case testing might be minimal or incomplete.
2. **Limited Complexity in Test Scenarios** : The testbench may not include high-level stimulus like concurrent transactions or randomization beyond simple input sequences.
3. **Exclusion of Post-Silicon Validation** : The verification is limited to simulation-level checks. Real-time performance, power consumption, and hardware-software integration are beyond the scope.
4. **Configuration and Protocol Edge Cases** : Certain rare or extreme APB configuration cases and edge protocol behaviors might not be included or validated due to prioritization of main flows.
5. **Lack of Assertions and Formal Verification** : The project focuses on directed and coverage-driven simulation; assertion-based verification (SVA) or formal methods are not part of the current framework.



Innovation

The innovation in this project lies in applying a modular **Universal Verification Methodology (UVM)** framework to validate a parameterized and configurable **Vending Machine Controller IP**, which is typically verified using simpler, directed testbenches. Instead of conventional approaches, this project leverages reusable UVM components such as sequences, drivers, monitors, and a scoreboard to create a scalable test environment. The ability to handle asynchronous currency inputs, configuration via APB interface, and item-dependent logic like dispensing and change return introduces realistic edge conditions, which are explored using both directed and constrained-random stimulus. Furthermore, the verification testbench is capable of tracking transaction-level activity and checking for functional correctness across multiple items and dynamic configuration scenarios. This not only demonstrates strong practical application of UVM in a real-world embedded system but also builds a foundation for future automation and coverage-driven enhancements.



Project Objectives

Project Objectives and Expected Outcomes

1. To verify the functional correctness of the Vending Machine Controller for various operations such as currency insertion, item selection, item dispensing, and change return.

Expected Outcome : A functionally verified Vending Machine Controller IP, with correct output behavior for all valid transaction flows.

2. To build a modular UVM-based testbench that includes reusable components like sequences, drivers, monitors, agents, scoreboards, and coverage models.

Expected Outcome : A reusable and scalable UVM verification environment, adaptable to future enhancements or different IPs with minimal modifications.

3. To validate control flow and mode transitions, ensuring the design moves correctly between Reset, Configuration, and Operation modes as defined in the specification.

Expected Outcome : Thoroughly tested state transitions and mode behaviors, ensuring robust control logic across different operational conditions.

4. To handle asynchronous currency input effectively in the testbench environment and verify correct synchronization with the system clock domain.

Expected Outcome : Correct handling of asynchronous input signals, accurately mimicking real-world vending machine interactions in a simulated environment.



5. To achieve complete functional coverage by writing diverse test cases and capturing corner scenarios through directed and random stimulus.

Expected Outcome : Comprehensive coverage reports and documented test cases, confirming that all design features and edge cases have been exercised.

Deliverables

1. Verified Vending Machine Controller IP

The final RTL module will be verified against the specification to ensure proper operation under all valid inputs, including item dispensing, change return, and error handling.

2. UVM-Based Testbench Architecture

This includes reusable components such as the driver, monitor, sequencer, scoreboard, and agent, structured in a modular way to support easy extension and reuse.

3. Test Case Suite and Functional Coverage Report

Test cases cover normal and edge-case behaviors, and the functional coverage report ensures that all significant design conditions have been exercised.

4. Documentation and User Guide

This ensures future users can understand, reuse, or build upon the verification framework and the Vending Machine Controller design.



Methodologies and Results

Methods/Technology Used

- **Verification Methodology:** SystemVerilog with UVM
- **Verification Strategy:** Combination of directed and constrained-random test scenarios
- **Modeling Technique:** FSM-based modeling for controller design
- **Simulation:** Functional simulation using EDA Playground(synopsys vcs)

Tools / Software Used

- **Design & Simulation :** EDA Playground (for UVM simulation)
- **Version Control :** Git and GitHub
- **Documentation :** MS Word

Data Collection Approach

- Simulation logs were generated for all test scenarios
- Functional coverage data collected through covergroups
- Scoreboard results and DUT outputs were monitored and compared in simulation



Project Architecture

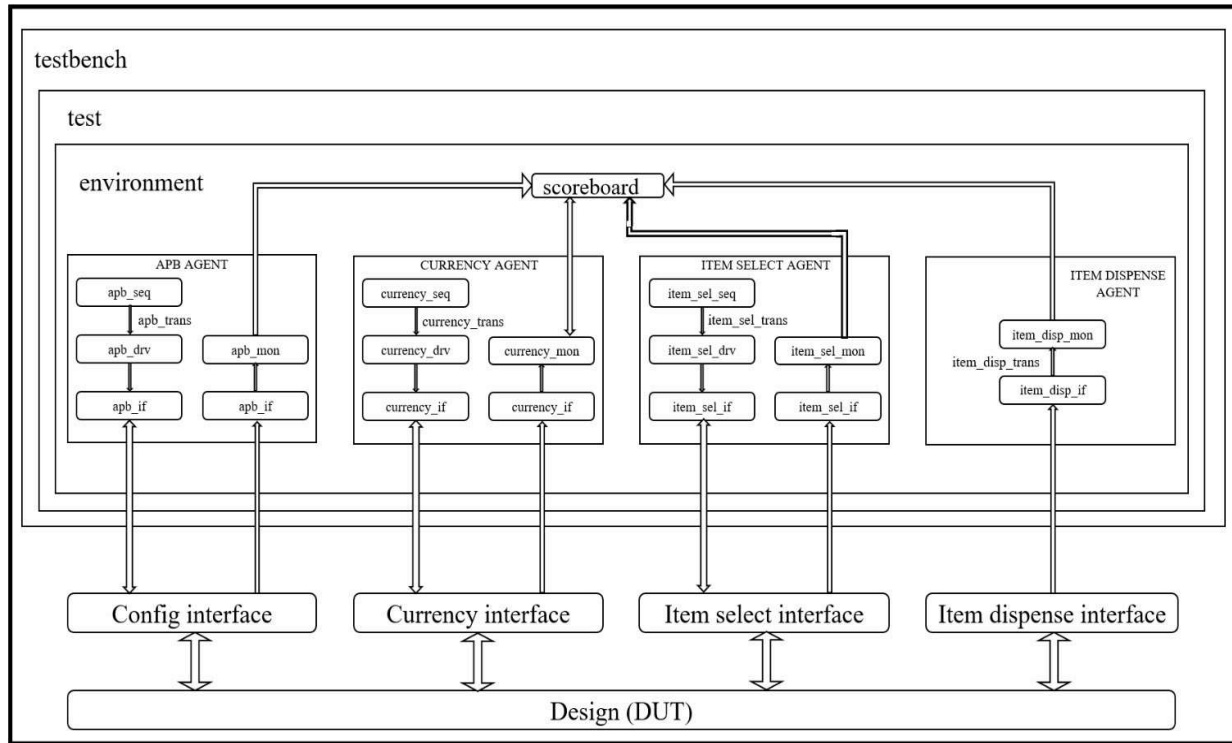


Figure 1 : UVM Architecture Block Diagram

This project follows a structured and modular verification flow using Universal Verification Methodology (UVM) to validate a configurable Vending Machine Controller IP. The methodology ensures that the design operates correctly across various real-world use cases, following the behavioral guidelines specified in the official design specification.

1. Understanding the Specification

The verification process began by thoroughly studying the Vending Machine Controller specification. Key design features were identified, including:



- Item selection interface
- Currency input (asynchronous)
- APB-based configuration mode
- Item dispensing and change return logic
- Mode transitions: Reset → Configuration → Operation

This understanding provided the foundation for modeling realistic verification scenarios.

2. UVM Testbench Architecture

A UVM-based testbench in Figure 1 was developed using a layered approach, with clearly defined and reusable components:

Test Layer (uvm_test): Controls which scenario is executed.

Environment (uvm_env): Acts as the container for all verification components.

Agents:

- **Currency Agent:** Handles asynchronous coin input transactions.
- **Item Selection Agent:** Sends selection codes and synchronizes with the system clock.
- **APB Configuration Agent:** Programs item values, counts, and resets during configuration mode.



Each agent includes a **driver**, **sequencer**, and **monitor** to isolate transaction generation, signal driving, and output observation.

3. Stimulus and Sequence Generation

Directed and scenario-based sequences were developed to verify the controller's response to valid and invalid operations:

- Inserting exact, excess, and insufficient currency.
- Selecting in-stock and out-of-stock items.
- Configuring items and values during APB-based configuration mode.
- Edge cases such as immediate selection after configuration or rapid insertions.

4. Scoreboard and Functional Coverage

Directed and scenario-based sequences were developed to verify the controller's response to valid and invalid operations:

A scoreboard was implemented to compare DUT outputs with reference outputs for each transaction.

Functional coverage points were defined to ensure all meaningful conditions were tested:

- All currency denominations (5, 10, 15, 20, 50, 100)
- Valid and invalid item selections
- Empty item response and change handling



- Configuration writes through APB interface

Coverage results were analyzed post-simulation to guide additional test creation and improve completeness.

5. Simulation and Debug

The complete testbench was compiled and simulated on supported EDA tools. Each scenario was observed using waveform viewers, and outputs were validated through both:

- Console logs and transaction reports
- Waveforms for signal-level debugging and timing checks

This methodology ensured that the controller operated according to specification and was ready for integration into a larger system.

Results

The Vending Machine Controller IP was successfully verified using a UVM based environment. The design met all functional and timing requirements, including dispensing items within 10 system clock cycles and handling asynchronous currency inputs reliably.



Final Project Hardware and Working Screenshots

```
# Log
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: UVM_INFO @ 0: reporter [RNTST] Running test vending_test...
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=8751, pldata=f4a66e8, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=8751, pldata=f4a66e8, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=6db6, pldata=c3aa7151, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=6db6, pldata=c3aa7151, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=3a98, pldata=2c6a8623, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=3a98, pldata=2c6a8623, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=eddf, pldata=498aa55e, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=eddf, pldata=498aa55e, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=8793, pldata=1b0acf02, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=8793, pldata=1b0acf02, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=7b4, pldata=a0eb030f, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=7b4, pldata=a0eb030f, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=6e42, pldata=db2b4185, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=6e42, pldata=db2b4185, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=bb3d, pldata=c9cb8a64, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=bb3d, pldata=c9cb8a64, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=eea5, pldata=6ccbdac, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=eea5, pldata=6ccbdac, pwrite=1
# KERNEL: UVM_INFO /home/runner/apb_seq.sv(15) @ 0: uvm_test_top.env.apb_agh.seqr@apb_seq [APB_SEQ] Transaction Generated: paddr=87a, pldata=c42c3b5d, pwrite=0
# KERNEL: UVM_INFO /home/runner/apb_drv.sv(49) @ 0: uvm_test_top.env.apb_agh.drv [APB_DRV] Transaction Generated: paddr=87a, pldata=c42c3b5d, pwrite=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 0: uvm_test_top.env.currency_agh.seqr@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 85
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 0: uvm_test_top.env.currency_agh.drv [CURRENCY_DRV] currency_select_valid:0 || currency_value: 85
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 15: uvm_test_top.env.item_sel_agh.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 15: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 15: uvm_test_top.env.item_disp_agh.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(42) @ 15: uvm_test_top.env.sb [ITEM_DISP_COREBOARD] Received Item Dispense: item= 0, change= 0
# KERNEL: UVM_INFO /home/runner/currency_mon.sv(31) @ 15: uvm_test_top.env.currency_agh.m [CURRENCY_MON] currency_select_valid:0 || currency_select: 85
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(35) @ 15: uvm_test_top.env.sb [CURRENCY_SCOREBOARD] Received Currency: value=85, valid=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 15: uvm_test_top.env.currency_agh.seqr@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 55
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 15: uvm_test_top.env.currency_agh.drv [CURRENCY_DRV] currency_select_valid:0 || currency_value: 55
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 35: uvm_test_top.env.item_sel_agh.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 35: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
```

Figure 2 : Test cases of project

Figure 2 The log file in EDA Playground shows that all assertions passed successfully, indicating correct simulation behavior. Each test case was executed, and the expected outputs matched the actual results. This confirms that the design under test (DUT) is functioning as intended across all scenarios, validating its logic and performance. and we successfully passed all test cases



```
Log Share
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 95: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 95: uvm_test_top.env.item_disp_agm.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(42) @ 95: uvm_test_top.env.sb [ITEM_DISP_COREBOARD] Received Item Dispense: item= 0, change= 0
# KERNEL: UVM_INFO /home/runner/currency_mon.sv(31) @ 95: uvm_test_top.env.currency_agm.m [CURRENCY_MON] currency_select_valid:0 || currency_select: 75
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(35) @ 95: uvm_test_top.env.sb [CURRENCY_SCOREBOARD] Received Currency: value=75, valid=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 95: uvm_test_top.env.currency_agm.currency_seq@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 80
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 95: uvm_test_top.env.currency_agm.d [CURRENCY_DRV] currency_select_valid:0 || currency_value: 80
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 115: uvm_test_top.env.item_sel_agm.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 115: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 115: uvm_test_top.env.item_disp_agm.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(42) @ 115: uvm_test_top.env.sb [ITEM_DISP_COREBOARD] Received Item Dispense: item= 0, change= 0
# KERNEL: UVM_INFO /home/runner/currency_mon.sv(31) @ 115: uvm_test_top.env.currency_agm.m [CURRENCY_MON] currency_select_valid:0 || currency_select: 80
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(35) @ 115: uvm_test_top.env.sb [CURRENCY_SCOREBOARD] Received Currency: value=80, valid=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 115: uvm_test_top.env.currency_agm.currency_seq@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 5
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 115: uvm_test_top.env.currency_agm.d [CURRENCY_DRV] currency_select_valid:0 || currency_value: 5
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 135: uvm_test_top.env.item_sel_agm.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 135: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 135: uvm_test_top.env.item_disp_agm.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(42) @ 135: uvm_test_top.env.sb [ITEM_DISP_COREBOARD] Received Item Dispense: item= 0, change= 0
# KERNEL: UVM_INFO /home/runner/currency_mon.sv(31) @ 135: uvm_test_top.env.currency_agm.m [CURRENCY_MON] currency_select_valid:0 || currency_select: 5
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(35) @ 135: uvm_test_top.env.sb [CURRENCY_SCOREBOARD] Received Currency: value=5, valid=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 135: uvm_test_top.env.currency_agm.currency_seq@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 5
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 135: uvm_test_top.env.currency_agm.d [CURRENCY_DRV] currency_select_valid:0 || currency_value: 5
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 155: uvm_test_top.env.item_sel_agm.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 155: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 155: uvm_test_top.env.item_disp_agm.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(42) @ 155: uvm_test_top.env.sb [ITEM_DISP_COREBOARD] Received Item Dispense: item= 0, change= 0
# KERNEL: UVM_INFO /home/runner/currency_mon.sv(31) @ 155: uvm_test_top.env.currency_agm.m [CURRENCY_MON] currency_select_valid:0 || currency_select: 5
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(35) @ 155: uvm_test_top.env.sb [CURRENCY_SCOREBOARD] Received Currency: value=5, valid=0
# KERNEL: UVM_INFO /home/runner/currency_seq.sv(17) @ 155: uvm_test_top.env.currency_agm.currency_seq@currency_seq [CURRENCY_SEQ] currency_select_valid:0 || currency_select_val: 40
# KERNEL: UVM_INFO /home/runner/currency_drv.sv(30) @ 155: uvm_test_top.env.currency_agm.d [CURRENCY_DRV] currency_select_valid:0 || currency_value: 40
# KERNEL: UVM_INFO /home/runner/item_sel_mon.sv(31) @ 175: uvm_test_top.env.item_sel_agm.m [ITEM_SEL_MON] item_select_valid:0 || item_select: 0
# KERNEL: UVM_INFO /home/runner/scoreboard.sv(39) @ 175: uvm_test_top.env.sb [ITEM_SEL_SCOREBOARD] Received Item Selection: item= 0, valid=0
# KERNEL: UVM_INFO /home/runner/item_disp_mon.sv(32) @ 175: uvm_test_top.env.item_disp_agm.m [ITEM_DISP_MON] item_select_valid:0 || item_select: 0 || currency_change : 0
```

Figure 3: Test cases of project

Figure 3 depicts The log file in EDA Playground shows that all assertions passed successfully, indicating correct simulation behavior. Each test case was executed, and the expected outputs matched the actual results. This confirms that the design under test (DUT) is functioning as intended across all scenarios, validating its logic and performance and we successfully passed all test cases.

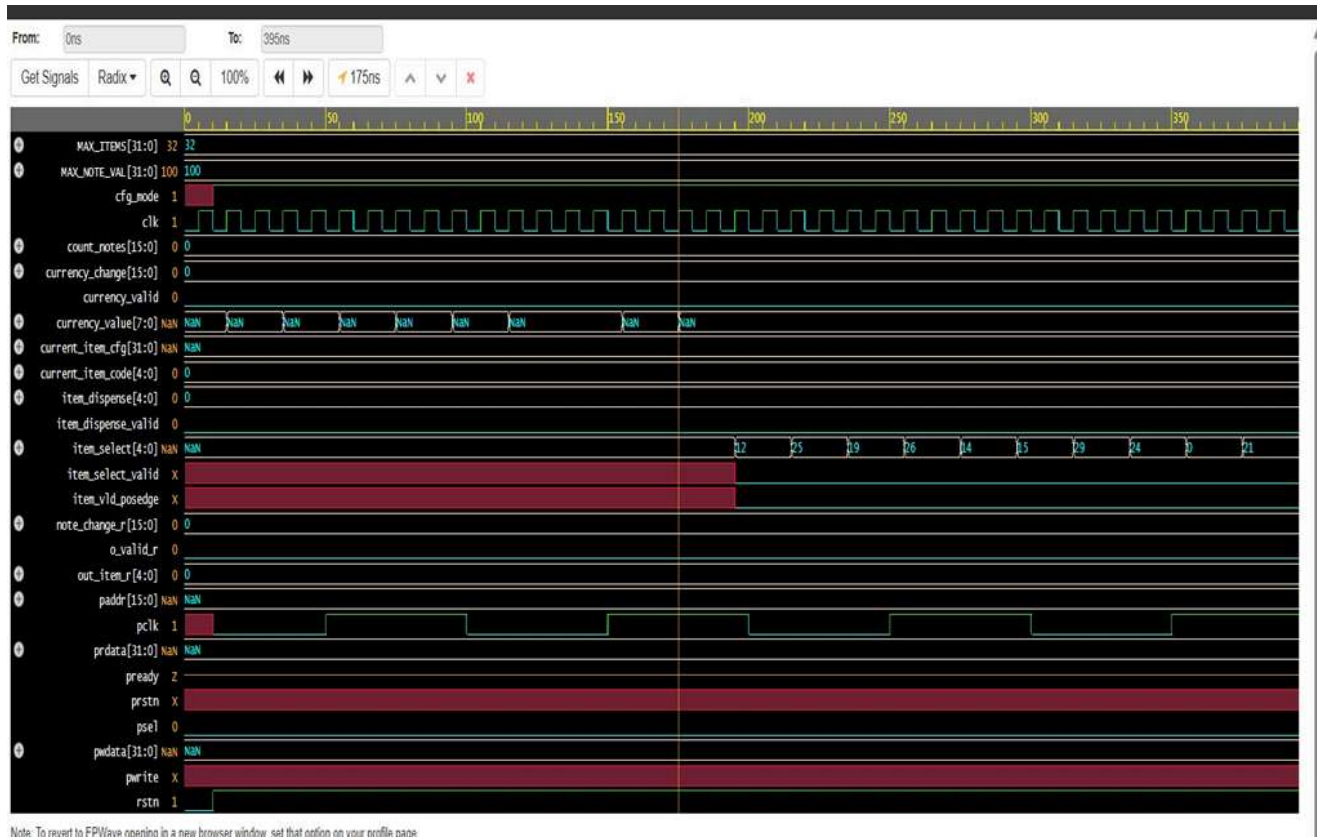


Figure 4: Wave forms of the project

Figure 4 depicts the waveforms in EDA Playground visually confirm the correct behavior of the design. Signal transitions align perfectly with expected outputs over time. Inputs trigger appropriate changes in outputs, verifying timing and logic. Combined with the log file, the waveforms provide strong evidence that all test cases pass accurately and consistently and we successfully achieve it.

GitHub Link

<https://github.com/sure-trust/Dhanush-H-g3-integrated-vlsi/tree/afcc0b398e3dbcd11a122bd9ef9f3317a35afb3/Final%20Capstone%20Project/April%2020th%20update>



Learning and Reflection

Learning and Reflection

This project provided each member with valuable technical and collaborative exposure, promoting both individual skill development and team synergy. The key learnings are summarized below:

Mr. Dhanush H (B.E ECE 4th Year) – Team Leader

- Designed configurable modules using parameterized Verilog, improving code reusability and scalability.
- Gained strong understanding of state machine design and testbench development through work on the item_dispense module.
- Managed team workflow through a task timeline, ensuring smooth integration of modules.
- Oversaw both design and verification activities, coordinating efforts across the team.
- Developed better communication and collaboration skills for project execution.

Mr. Shankar J (B.Tech ECE Graduate – 2024)

- Understood the concept and implementation of a Finite State Machine (FSM)-based Vending Machine Controller.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- Gained experience in coding with Verilog HDL and using the APB protocol for peripheral communication.
- Collaborated with the team to write reusable sequences and functional test cases and Developed and verified APB agent modules with transaction-level modeling

Ms. Sailaja G (B.Tech DEPARTMENT Graduate – 2024)

- Gained exposure to testbench component development including drivers and monitors.
- Participated in functional coverage planning and scoreboard development.
- Simulated and debugged testbench behavior to ensure DUT compliance.
- Maintained documentation and ensured test cases aligned with the verification plan.

Mr. Saiteja B (B.Tech ECE Graduate – 2024)

- Coordinated team efforts in developing a layered UVM testbench architecture.
- Designed the configuration module and created directed and random test cases.
- Gained practical skills in functional verification methodology and self-checking testbenches.

Experience

Overall, it was a great learning experience where we grew together as a team, improved our technical skills, and learned the value of coordination, problem-solving, and collaboration in completing a successful hardware project.



Conclusion and Future Scope

Objectives

The primary objective of this project was to design a verification environment for a parameterized and configurable Vending Machine Controller IP using Universal Verification Methodology (UVM). The goals included:

- Verifying correct functionality for various operations such as currency input, item selection, dispensing, and change return.
- Developing a modular, reusable UVM testbench.
- Validating FSM-like behavior and handling asynchronous interfaces.
- Collecting meaningful functional coverage to ensure all scenarios were tested.

Achievements

- Successfully developed a UVM-based verification environment with structured components like sequences, drivers, monitors, scoreboard, and coverage models.
- Simulated and verified the controller's behavior across different operational modes (Reset, Configuration, Operation).



- Handled asynchronous input domains, such as currency insertion, by creating appropriate interface synchronization in the testbench.
- Created a functional coverage-driven test suite to validate a wide variety of transaction flows and corner cases.
- Delivered complete documentation and a maintainable simulation flow for future reference and reuse.

Conclusion

This project demonstrated a practical implementation of UVM for verifying a realistic and configurable digital system. By creating a layered, reusable verification environment, the team gained deep insight into both RTL behavior and verification methodologies. The results confirmed that the controller IP works as intended under a variety of real-world scenarios, fulfilling all functional requirements from the specification. The project also emphasized the importance of coverage analysis and modularity in modern verification flows.

Future Scope

- Add SystemVerilog Assertions (SVA): To enhance reliability and catch protocol violations through inline assertion checks.
- Integrate Scoreboard Enhancements: Extend scoreboard logic to track complex transactions and detect mismatches with higher granularity.
- Support for Randomized and Constrained Inputs: Expand input generators to include intelligent constrained-random testing for stress testing.



- Formal Verification Integration: Apply formal tools to complement simulation, ensuring exhaustive checks for deadlocks and unreachable states.
- Visualization and GUI Testing: Link the testbench to waveform viewers and possibly a GUI front-end to mimic actual vending machine user interaction.